Python for Data Science

Course Plan

For Complete Beginners → Data Science Ready

Zero to Advanced | Pre-Class & Post-Class Activities | Overview of In-class topics

DAY 1 — Introduction to Python & Environment Setup

Objective: You will be learning what Python is, why it is used in Data Science, and can write and run their first Python program.

PRE-CLASS (Self Study)	IN-CLASS (The topics we will be covering)	POST-CLASS (Assignment)
• What is Programming? • What is Python? • Read: Why Python for Data Science • Install Python on your system (Windows/Linux/Mac) and environment tools : ➢ Anaconda Distribution ➢ Jupyter Notebook ➢ VS Code ➢ Git + GitHub account ➢ Kaggle account ➢ Create a free Google account for Google Colab	• Introduction to Programming Concepts • What is Python? • Why Python for Data Science? • Python vs Other Languages (brief) • Python Applications in Real World • Tour: Jupyter Notebook • Tour: Google Colab • Running First Python Program • print() Statement • Comments (# single line, ''' multiline) • Variables – Declaration & Naming Rules • Data Types – int, float, str, bool, NoneType • type() Function • Input and Output – input() • Type Casting – int(), float(), str() • Arithmetic Operators (+, -, *, /, //, %, **) • Comparison Operators (==, !=, >, <, >=, <=) • Logical Operators (and, or, not) • Assignment Operators (+=, -=, *=)	• Install Python + PyCharm • Write a program that takes your name and age, prints: Hello [Name], you will be [X] in 5 years • Practice: 5 variables of different data types, print each with type() • Practice: Type casting – convert string to int, float to int • Practice 10 print() exercises with different data types • Assignment: Mini Bio Program – Name, Age, City, Hobby as variables → print formatted output • Read: Python official docs – Built-in Types (overview only)

DAY 2 — Python Basics: Operators, Strings, Collections & Conditionals

Objective: You will be learning core Python data structures, operators, and how to make decisions with conditional logic.

PRE-CLASS (Self Study)	IN-CLASS (The topics we will be covering)	POST-CLASS (Assignment)
• Revise: Variables and Data Types from Day 1 • Read: Python String Methods (docs.python.org) • Get an overview on what are Lists and Dictionaries in python.	• Strings – Definition and Creation • String Indexing and Slicing • String Methods (upper, lower, strip, split, replace, find, len) • f-Strings (Formatted String Literals) • Lists – Creation, Indexing, Slicing • List Methods (append, remove, pop, sort, reverse, len) • List Comprehensions (Introduction) • Tuples – Immutable Lists, Packing & Unpacking • Dictionaries – Key-Value Pairs, get(), update() • Dictionary Comprehensions (Introduction) • Sets and Frozensets – Unique Elements • Set Operations (union, intersection, difference) • Conditional Statements – if, elif, else • Nested Conditions • Ternary / Inline if	• Practice: 5 programs using arithmetic and comparison operators • String Exercises: Reverse a string, count vowels, check palindrome • List Exercises: Create a grocery list – add, remove, sort, print items • Dictionary Exercise: Student record with name, age, marks – print formatted • Set Exercise: Find common elements between two lists using sets • Conditional Exercise: Grade calculator – take marks as input, print A/B/C/D/F • Mini Project: Simple Calculator using if-elif-else • Comprehension Practice: Create a list of squares of numbers 1-20 using list comprehension

DAY 3 — Loops, Functions & Modules

Objective: Students master loops for repetition, write reusable functions, and learn to use Python modules.

PRE-CLASS (Self Study)	IN-CLASS (The topics we will be covering)	POST-CLASS (5 Hours Assignment)
• Revise: Lists and Dictionaries from Day 2 • Read: Python for loop documentation • Get an overview on Python Functions and Python Modules and pip	• for Loop – Iterating over sequences • range() Function • while Loop • break, continue, pass • Nested Loops • Loop with else clause • Functions – def keyword • Parameters and Arguments • Default Arguments • Keyword Arguments • *args and **kwargs • Return Statements • Scope – Local vs Global Variables • Lambda Functions • map(), filter(), zip() • Modules – import, from...import, as • Built-in Modules: math, os, sys • random Module (random, randint, choice, shuffle) • datetime Module (date, time, timedelta) • pip – Package Manager • Installing Packages with pip	• Loop Exercises: Print multiplication table for any number using for loop • Loop Exercise: Print all prime numbers between 1-100 • Function Exercise: Write a function to check if number is even/odd • Function Exercise: Fibonacci sequence using a function • Function Exercise: Factorial using recursion • Lambda Exercise: Use lambda with map() to square all numbers in a list • Module Exercise: Use random module to create a dice rolling simulator • Module Exercise: Use datetime to show how many days until New Year • pip Exercise: Install the colorama package and use it to print colored text • Mini Project: Number Guessing Game using random + while loop + functions

DAY 4 — Advanced Python: OOP, File Handling, Exception Handling & More

Objective: Students learn Object-Oriented Programming, file I/O, error handling, and advanced Python concepts used in real-world Data Science projects.

PRE-CLASS (Self Study)	IN-CLASS (The topics we will be covering))	POST-CLASS (Assignment)
• Read: What is Object-Oriented Programming? (concept article) • Get an overview on Python Classes & Objects • Read: Python File Handling Basics • Read: Python Exception Handling	• File Handling – open(), read(), write(), append(), close() • with Statement (Context Manager) • Reading Line by Line • Working with JSON – json.load(), json.dump(), json.loads(), json.dumps() • Classes and Objects – class keyword • Constructors – __init__() method • Instance Variables and Methods • self keyword • Inheritance – Single and Multiple • Method Overriding • Operator Overloading (__add__, __str__, etc.) • Encapsulation and Abstraction (concept) • __main__ and if __name__ == '__main__' • Exception Handling – try, except, else, finally • Common Exceptions (ValueError, TypeError, FileNotFoundError) • Custom Exceptions – raise • Generators – yield keyword • Iterators – __iter__, __next__ • Decorators – @decorator syntax	• File Exercise: Read a text file, count words, lines, and characters • JSON Exercise: Create a student database in JSON, read and update it • OOP Exercise: Create a BankAccount class with deposit, withdraw, balance methods • OOP Exercise: Create Animal class, then Dog and Cat subclass with speak() method • Exception Exercise: File reader with full exception handling (file not found, permission error) • Generator Exercise: Write a generator for Fibonacci sequence • Decorator Exercise: Write a timing decorator that prints execution time of a function • Debug Exercise: Fix 5 buggy Python programs using PyCharm debugger • Mini Project: Student Management System using classes, file handling, and JSON

DAY 5 — NumPy for Data Science

Objective: Students understand NumPy arrays and use them for fast numerical computation, the foundation of Data Science in Python.

PRE-CLASS (Self Study)	IN-CLASS (The topics we will be covering)	POST-CLASS (Assignment)
• Read: What is NumPy and why is it faster than Python lists? • Get an overview on NumPy Introduction • Open Google Colab – ready to practice	• Introduction to NumPy – What & Why • NumPy vs Python Lists (Performance, Memory) • Importing NumPy – import numpy as np • Creating Arrays – np.array(), np.zeros(), np.ones(), np.arange(), np.linspace() • Array Attributes – shape, ndim, size, dtype • 1D, 2D, 3D Arrays • Array Indexing – Positive and Negative • Array Slicing – 1D and 2D • Boolean Indexing • Fancy Indexing • Basic Array Operations (+, -, *, /, **) • Element-wise Operations • Broadcasting • Aggregation Functions – sum, mean, min, max, std, var • Axis-wise Operations • Matrix Operations – dot(), matmul(), transpose() • Reshaping – reshape(), flatten(), ravel() • Stacking – vstack(), hstack(), concatenate() • Sorting and Searching • Random Module in NumPy – np.random • Loading Data with NumPy – np.loadtxt(), np.genfromtxt()	• Array Creation: Create 1D, 2D, and 3D arrays using different methods • Slicing Exercise: Extract sub-arrays from a 5x5 matrix (various patterns) • Boolean Indexing: From an array of 20 random numbers, extract values > 50 • Broadcasting Exercise: Add a 1D array to each row of a 2D array • Aggregation: Create a 4x4 matrix, find sum, mean, min, max column-wise and row-wise • Matrix Multiplication: Multiply two matrices manually then verify with np.dot() • Reshape Exercise: Create a 1D array of 24 elements, reshape into 2x3x4 • Stacking Exercise: Stack 3 arrays horizontally and vertically • Mini Project: Analyze student marks using NumPy – find topper, average, failing students • Read: NumPy official documentation – Array creation routines

DAY 6 — Pandas for Data Science

Objective: Students learn Pandas for real-world data loading, cleaning, transformation, and exploration – the most-used tool in Data Science.

PRE-CLASS (Self Study)	IN-CLASS (1 Hour with Teacher)	POST-CLASS (Assignment)
• Download a sample CSV dataset (e.g., Titanic dataset from Kaggle) • Read: What is Pandas? • Read: Pandas Series vs DataFrame	• Introduction to Pandas – What & Why • Series – Creation, Indexing, Operations • DataFrame – Creation from dict, list, CSV • DataFrame Attributes – shape, columns, dtypes, info() • head(), tail(), describe() • Reading CSV – pd.read_csv() • Reading Excel – pd.read_excel() • Writing to CSV and Excel • Selecting Columns and Rows – , loc, iloc • Filtering Data – Boolean Conditions • Sorting – sort_values() • Handling Missing Values – isnull(), dropna(), fillna() • Adding and Removing Columns • Renaming Columns – rename() • Group By – groupby(), agg() • Concat – pd.concat() • Merge – pd.merge() (inner, outer, left, right joins) • Data Cleaning Techniques • Data Transformation – apply(), map(), applymap() • String Operations in Pandas – .str methods • Datetime in Pandas – pd.to_datetime() • Exploratory Data Analysis (EDA) – value_counts(), corr(), nunique()	• Series Exercise: Create a Series of monthly sales, compute % change • CSV Exercise: Load Titanic dataset, display shape, info, describe • Filtering Exercise: From Titanic, filter all female passengers who survived • Missing Values: Find columns with nulls, fill with mean/mode/drop • GroupBy Exercise: Average fare by passenger class • Merge Exercise: Create two DataFrames (students + marks), merge on student ID • Data Cleaning: Fix inconsistent column names, remove duplicates • Transformation: Apply a custom function to convert grades to letter grade using apply() • EDA Exercise: Find top 5 most common values in a categorical column • Mini Project: Full EDA on a dataset of your choice – 10 insights with code

DAY 7 — Data Visualization (Matplotlib & Seaborn) + APIs

Objective: Students visualize data effectively using Matplotlib and Seaborn, and learn to fetch real-world data using APIs.

PRE-CLASS (Self Study)	IN-CLASS (1 Hour with Teacher)	POST-CLASS (Assignment)
• Install libraries: pip install matplotlib seaborn requests • Watch: Matplotlib Basics Tutorial (YouTube – 15 min) • Watch: What is an API? (YouTube – 5 min) • Read: What is REST API? (beginner article)	• Why Data Visualization? • Introduction to Matplotlib • plt.plot() – Line Chart • plt.bar() – Bar Chart • plt.barh() – Horizontal Bar • plt.pie() – Pie Chart • plt.hist() – Histogram • plt.scatter() – Scatter Plot • Adding Labels, Titles, Legends • Subplots – plt.subplot() • Figure Size and DPI • Saving Plots – savefig() • Introduction to Seaborn • sns.heatmap() – Correlation Heatmap • sns.pairplot() • sns.boxplot() • sns.violinplot() • sns.countplot() • sns.lineplot() and sns.barplot() • Seaborn Themes and Color Palettes • Data Exploration Through Visualization • Insights Generation from Charts • Introduction to APIs – REST, HTTP Methods • requests Library – get(), post() • API Response – status_code, json() • Calling Public APIs (weather, news, exchange rate) • Working with JSON API Responses • API Keys and Authentication Basics	• Matplotlib Exercise: Create line, bar, pie, histogram, scatter for a sales dataset • Subplot Exercise: Display all 5 chart types in one figure using subplots • Seaborn Exercise: Heatmap of correlation matrix from Titanic dataset • Seaborn Exercise: Pairplot and countplot on any dataset • Visualization Project: Choose a dataset, create 5 meaningful charts with titles and labels • API Exercise: Call OpenWeatherMap API for your city – display temperature and weather • API Exercise: Call a free news API, extract top 5 headlines and print them • JSON Exercise: Parse a nested JSON API response and extract specific fields • Mini Project: Weather Dashboard – fetch live data from API, display using Matplotlib

DAY 8 — Testing, Databases, Streamlit, FastAPI , Git & GitHub Basics

Objective: Students learn automated testing, database basics, and build a simple Data Science web app using Streamlit and FastAPI – completing the full Python Data Science stack. Git & GitHub Basics – Version Control

PRE-CLASS (Self Study)	IN-CLASS (1 Hour with Teacher)	POST-CLASS (5 Hours Assignment)
• Read: What is Unit Testing? • Watch: Streamlit in 10 Minutes • Read :FastAPI Introduction • Read : What is a Database? SQL Basics	• Introduction to Testing – Why Test? • Automated Testing with pytest • Writing Test Functions • Running Tests – pytest command • Assertions in Tests • Data Validation with Pydantic – BaseModel, Field • Introduction to Databases • Database CRUD Operations – Create, Read, Update, Delete • Git & GitHub Basics – Version Control • Markdown in Jupyter Notebooks • Introduction to Streamlit • st.title(), st.write(), st.text_input() • st.dataframe(), st.chart() • st.sidebar, st.button, st.selectbox • Running Streamlit App – streamlit run app.py • Introduction to FastAPI • Creating API Endpoints – @app.get(), @app.post() • Request and Response Models with Pydantic • Running FastAPI – uvicorn main:app • Analytics App – FastAPI Backend + Streamlit Frontend • Complete Python for Data Science Stack Review	• Pytest Exercise: Write unit tests for your Day 3 functions (even/odd, factorial, Fibonacci) • Pydantic Exercise: Create a StudentModel with name, age, marks with validation • SQLite Exercise: Create a database, insert student records, query and display • Streamlit Exercise: Build a simple data explorer – upload CSV, display data and charts • FastAPI Exercise: Build an API with endpoints to get/add student data • Integration Exercise: Connect FastAPI backend + Streamlit frontend • CAPSTONE PROJECT: Build a Data Science Web App • → Load a CSV dataset • → Clean and analyze data with Pandas • → Visualize with Matplotlib/Seaborn • → Expose insights via FastAPI endpoint • → Display on Streamlit dashboard • Prepare a short presentation (5 slides) explaining your project

Quick Reference — 8-Day Overview

Day	Topic Focus	Key Skills Gained
Day 1	Python Setup & Basics	Environment, Variables, Data Types, I/O
Day 2	Operators, Collections & Conditionals	Strings, Lists, Dict, Sets, if-elif-else
Day 3	Loops, Functions & Modules	for/while, Functions, Lambda, pip, datetime
Day 4	Advanced Python & OOP	Classes, File I/O, JSON, Decorators, Generators
Day 5	NumPy	Arrays, Slicing, Broadcasting, Matrix Ops
Day 6	Pandas	DataFrames, EDA, GroupBy, Merge, Data Cleaning
Day 7	Visualization & APIs	Matplotlib, Seaborn, requests, REST APIs
Day 8	Testing, DB, Streamlit, FastAPI, Version Control	pytest, SQLite, Streamlit App, FastAPI Backend